SIMATS ENGINEERING

SAVEETHA SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL – 3
CODING CHALLENGE

Course Code	**DSA0135**	**Course Title**	**Object Oriented Programming for Real Time Applications**
CO Assessed	CO2 (BL3)	**Assessment**	Tool – 3
Weightage	10%	**Total Marks**	100
Student Name		**Reg No**	
Section / Batch		**Date**	
CO Statement	Design C++ programs by applying functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to solve real-world programming problems. (BL3)		

RUBRICS FOR EVALUATION

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1) A university aims to develop a Student Analytics System to evaluate academic performance across multiple departments, where each department has a varying number of students and their marks are stored in a two-dimensional structure. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage student data effectively.
The program should compute the topper in each department, identify the overall university topper, and calculate the pass percentage for each department.
Additionally, the solution must handle variable department sizes, edge cases such as empty departments, equal marks, or all students failing, and ensure efficient comparison across departments while maintaining optimized time complexity and clean modular design.

Program:

```
#include<iostream>
using namespace std;
class Student
{
private:
   int mark;
public:
   static int totalStudents;
   void input()
   {
      cin>>mark;
      totalStudents++;
   }
   int getMark()
   {
      return mark;
   }
};
int Student::totalStudents=0;
class StudentAnalytics
{
public:
   void inputData(Student s[][50],int deptSize[],int d)
   {
      for(int i=0;i<d;i++)
      {
         cout<<"Enter marks for Department "<<i+1<<":"<<endl;

         for(int j=0;j<deptSize[i];j++)
         {
            s[i][j].input();
         }
      }
   }
   void departmentAnalysis(Student s[][50],int deptSize[],int d,int topper[])
   {
      for(int i=0;i<d;i++)
      {
         if(deptSize[i]==0)
         {
```

```
            cout<<"Department "<<i+1<<" is Empty"<<endl;
            topper[i]=0;
            continue;
        }
        int top=s[i][0].getMark();
        int pass=0;
        for(int j=0;j<deptSize[i];j++)
        {
            if(s[i][j].getMark()>top)
                top=s[i][j].getMark();
            if(s[i][j].getMark()>=40)
                pass++;
        }
        topper[i]=top;
        double passPercentage=(pass*100.0)/deptSize[i];
        cout<<"\nDepartment "<<i+1<<endl;
        cout<<"Topper Mark: "<<top<<endl;
        cout<<"Pass Percentage: "<<passPercentage<<"%"<<endl;
    }
}
void universityTopper(int topper[],int d)
{
    int overall=topper[0];
    for(int i=1;i<d;i++)
    {
        if(topper[i]>overall)
            overall=topper[i];
    }
    cout<<"\nUniversity Topper Mark: "<<overall<<endl;
}
void report()
{
    cout<<"Total Students: "<<Student::totalStudents<<endl;
}
};
int main()
{
    int d;
    cout<<"Enter Number of Departments: ";
    cin>>d;
    Student s[10][50];
    int deptSize[10];
    int topper[10];
    for(int i=0;i<d;i++)
    {
        cout<<"Enter Number of Students in Department "<<i+1<<": ";
        cin>>deptSize[i];
    }
    StudentAnalytics sa;
    sa.inputData(s,deptSize,d);
    sa.departmentAnalysis(s,deptSize,d,topper);
    sa.universityTopper(topper,d);
    sa.report();
    return 0;
}
```

Test Case 1 – Normal Case
Input
3
4
3
5
78 85 90 67
45 55 35
60 75 80 95 40
Output
Department 1
Topper Mark: 90
Pass Percentage: 100%

Department 2
Topper Mark: 55
Pass Percentage: 66.6667%

Department 3
Topper Mark: 95
Pass Percentage: 100%

University Topper Mark: 95
Total Students: 12

Test Case 2 – Empty Department
Input
3
4
0
3
70 80 90 60
35 20 45
Output
Department 1
Topper Mark: 90
Pass Percentage: 100%

Department 2 is Empty

Department 3
Topper Mark: 45
Pass Percentage: 33.3333%

University Topper Mark: 90
Total Students: 7

Test Case 3 – All Students Fail
Input
2

3
4
20 30 35
10 25 15 39
Output
Department 1
Topper Mark: 35
Pass Percentage: 0%

Department 2
Topper Mark: 39
Pass Percentage: 0%

University Topper Mark: 39
Total Students: 7

Test Case 4 – Equal Topper Marks
Input
2
3
3
90 90 80
90 85 70
Output
Department 1
Topper Mark: 90
Pass Percentage: 100%

Department 2
Topper Mark: 90
Pass Percentage: 100%

University Topper Mark: 90
Total Students: 6

Test Case 5 – Single Department
Input
1
5
40 50 60 70 80
Output
Department 1
Topper Mark: 80
Pass Percentage: 100%

University Topper Mark: 80
Total Students: 5

Sample Input

```
3
4
3
5

78 85 90 67
45 55 35
60 75 80 95 40
```

Your Output

```
Enter Number of Departments: Enter Number of Students in Department 1: Enter Number o
Enter marks for Department 2:
Enter marks for Department 3:

Department 1
Topper Mark: 90
Pass Percentage: 100%
```

---

2. A country conducts elections across multiple regions, where votes for different candidates are stored in a two-dimensional structure representing regions and candidates. Develop a C++ program using object-oriented principles and functions to store and process vote data efficiently.
   A. The program should determine the winner in each region, compute the overall winner by aggregating votes across all regions, and detect tie conditions at both regional and overall levels.
   B. The implementation should use appropriate access specifiers, member functions, and static members where necessary, and must handle large input sizes efficiently. Additionally, the program should address edge cases such as ties, zero votes, and invalid inputs, while ensuring accurate multi-level comparisons and optimized performance.

   Program:

```
#include<iostream>
using namespace std;
class Election
{
private:
   int votes[10][10];
public:
   static int totalVotes;
   void input(int r,int c)
   {
      for(int i=0;i<r;i++)
```

```
    {
        for(int j=0;j<c;j++)
        {
            cin>>votes[i][j];
            if(votes[i][j]<0)
                votes[i][j]=0;
            totalVotes+=votes[i][j];
        }
    }
}
void regionalWinner(int r,int c,int total[],string region[],string candidate[])
{
    for(int i=0;i<r;i++)
    {
        int maxVote=votes[i][0];
        int winner=0;
        bool tie=false;
        for(int j=1;j<c;j++)
        {
            if(votes[i][j]>maxVote)
            {
                maxVote=votes[i][j];
                winner=j;
                tie=false;
            }
            else if(votes[i][j]==maxVote)
            {
                tie=true;
            }
        }
        if(tie)
            cout<<region[i]<<" : Tie"<<endl;
        else
            cout<<region[i]<<" Winner: "<<candidate[winner]<<endl;
        for(int j=0;j<c;j++)
            total[j]+=votes[i][j];
    }
}
void overallWinner(int total[],int c,string candidate[])
{
    int maxVote=total[0];
    int winner=0;
    bool tie=false;
    for(int i=1;i<c;i++)
    {
        if(total[i]>maxVote)
        {
            maxVote=total[i];
            winner=i;
            tie=false;
```

```
            }
            else if(total[i]==maxVote)
            {
               tie=true;
            }
        }
        if(tie)
            cout<<"Overall Result: Tie"<<endl;
        else
            cout<<"Overall Winner: "<<candidate[winner]<<endl;
    }
};
int Election::totalVotes=0;
int main()
{
    int r,c;
    cin>>r>>c;
    string region[10];
    string candidate[10];
    int total[10]={0};
    cout<<"Enter Region Names:"<<endl;
    for(int i=0;i<r;i++)
        cin>>region[i];
    cout<<"Enter Candidate Names:"<<endl;
    for(int i=0;i<c;i++)
        cin>>candidate[i];
    cout<<"Enter Votes:"<<endl;
    Election e;
    e.input(r,c);
    e.regionalWinner(r,c,total,region,candidate);
    e.overallWinner(total,c,candidate);
    cout<<"Total Votes: "<<Election::totalVotes;
    return 0;
}
```

Test Case 1 – Normal Case

Input

3 3
North Chennai South
Rahul Priya Arjun
100 80 60
50 90 70
120 110 100

Output

North Winner: Rahul
Chennai Winner: Priya
South Winner: Rahul
Overall Winner: Rahul
Total Votes: 780

Test Case 2 – Regional Tie
Input
2 3
North South
Rahul Priya Arjun
100 100 50
80 60 40
Output
North : Tie
South Winner: Rahul
Overall Winner: Rahul
Total Votes: 430

Test Case 3 – Overall Tie
Input
2 2
North South
Rahul Priya
100 50
50 100
Output
North Winner: Rahul
South Winner: Priya
Overall Result: Tie
Total Votes: 300

Test Case 4 – All Zero Votes
Input
2 3
North South
Rahul Priya Arjun
0 0 0
0 0 0
Output
North : Tie
South : Tie
Overall Result: Tie
Total Votes: 0

Test Case 5 – Negative Votes (Invalid Input)
Input
2 3
North South
Rahul Priya Arjun
-10 50 20
30 -5 40
Output
North Winner: Priya
South Winner: Arjun
Overall Winner: Priya

Total Votes: 140

Explanation

- -10 becomes 0
- -5 becomes 0

Votes become:
0 50 20
30 0 40

Test Case 6 – Single Region

Input

1 3
North
Rahul Priya Arjun
100 120 90

Output

North Winner: Priya
Overall Winner: Priya
Total Votes: 310

Test Case 7 – Single Candidate

Input

3 1
North Chennai South
Rahul
100
200
150

Output

North Winner: Rahul
Chennai Winner: Rahul
South Winner: Rahul
Overall Winner: Rahul
Total Votes: 450

Sample Input

```
3 3
North Chennai South
Rahul Priya Arjun
100 80 60
0 0 0
120 110 100
```

Your Output

```
Enter Region Names:
Enter Candidate Names:
Enter Votes:
North Winner: Rahul
Chennai : Tie
South Winner: Rahul
Overall Winner: Rahul
Total Votes: 570
```

---

3. In this scenario, a Retail Chain Management System is designed to analyze product sales across multiple stores using object-oriented programming in C++.

A. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage product data effectively. Each store contains a collection of products, and every product has attributes such as product name, price, and quantity sold and compute individual product sales (price × quantity), and display the information. An array of objects is used to store multiple products for each store, allowing the program to scale efficiently for different store sizes.

B. To manage multiple stores, the program can either use a two-dimensional array of objects or create another class such as Store, which contains an array of Product objects. For each store, the total sales are calculated by iterating through all its products and summing their individual sales using a dedicated member function like calculateStoreSales(). A static data member is used to maintain the overall total sales across all stores, demonstrating the use of shared class-level data. This helps in tracking cumulative performance of the entire retail chain.

C. The program also includes logic to identify the best-performing store, which is the store with the highest total sales, and the best-selling product, which is the product generating the highest revenue across all stores.

D. Proper input/output handling is essential, where the user is prompted to enter the number of stores, number of products per store, and corresponding product details. The output should be well-structured, displaying store-wise sales, overall sales, best-performing store, and top product.

Program:

```
#include<iostream>
using namespace std;
class Product
{
private:
   string name;
   float price;
   int qty;
public:
   static float totalSales;
   void input()
   {
      cin>>name>>price>>qty;
   }
   float calculateSales()
   {
      return price*qty;
   }
   string getName()
   {
      return name;
   }
};
float Product::totalSales=0;
int main()
{
   int stores,products;
   cin>>stores;
   Product p[10][10];
   float bestStoreSales=0;
```

```
    int bestStore=0;
    float bestProductSales=0;
    string bestProduct;
    for(int i=0;i<stores;i++)
    {
        cin>>products;
        float storeSales=0;
        for(int j=0;j<products;j++)
        {
            p[i][j].input();
            float sales=p[i][j].calculateSales();
            storeSales+=sales;
            Product::totalSales+=sales;
            if(sales>bestProductSales)
            {
                bestProductSales=sales;
                bestProduct=p[i][j].getName();
            }
        }
        cout<<"Store "<<i+1<<" Sales: "<<storeSales<<endl;
        if(storeSales>bestStoreSales)
        {
            bestStoreSales=storeSales;
            bestStore=i+1;
        }
    }
    cout<<"Overall Sales: "<<Product::totalSales<<endl;
    cout<<"Best Store: Store "<<bestStore<<endl;
    cout<<"Best Product: "<<bestProduct<<endl;
    return 0;
}
```

Test Case 1 – Normal Case

Input

2

3

Laptop 50000 2

Mobile 25000 5

Mouse 500 10

2

TV 30000 2

AC 40000 1

Output

Store 1 Sales: 230000

Store 2 Sales: 100000

Overall Sales: 330000

Best Store: Store 1

Best Product: Mobile

Test Case 2 – Single Store

Input

1

3

Laptop 50000 1

Mouse 500 10

Keyboard 1000 5

Output

Store 1 Sales: 60000

Overall Sales: 60000

Best Store: Store 1

Best Product: Laptop

Test Case 3 – Single Product Per Store

Input

3

1

Laptop 50000 2

1

Mobile 25000 5

1

TV 30000 3

Output

Store 1 Sales: 100000

Store 2 Sales: 125000

Store 3 Sales: 90000

Overall Sales: 315000

Best Store: Store 2

Best Product: Mobile

Test Case 4 – Product With Zero Quantity

Input

2

2

Laptop 50000 0

Mouse 500 10

2

TV 30000 1

AC 40000 0

Output

Store 1 Sales: 5000

Store 2 Sales: 30000

Overall Sales: 35000

Best Store: Store 2

Best Product: TV

Test Case 5 – Equal Product Sales

Input

1

2

Laptop 50000 2

Mobile 25000 4

Output

Store 1 Sales: 200000

Overall Sales: 200000

Best Store: Store 1

Best Product: Laptop

Explanation:

- Laptop = 100000
- Mobile = 100000
- Since the code uses > and not >=, the first product remains the best product.

Test Case 6 – Multiple Stores

Input

3

2

Laptop 50000 2

Mouse 500 10

3

Mobile 25000 5

TV 30000 2

Fan 2000 5

2

AC 40000 2

Keyboard 1000 10

Output

Store 1 Sales: 105000

Store 2 Sales: 195000

Store 3 Sales: 90000

Overall Sales: 390000

Best Store: Store 2

Best Product: Mobile

Sample Input

```
2 3
Laptop 50000 2
Mobile 25000 5
Mouse 500 10
TV 30000 2
AC 40000 1
Fan 2000 5
```

Your Output

```
Store 1 Sales: 230000
Store 2 Sales: 110000
Total Sales: 340000
Best Product: Mobile
```